The Language of Complaints

NLP-Powered NYC 311 Service Request Triage and Resolution Time Prediction

CSGY-6513 Big Data
Section D, Spring 2026

Prof. Amit Patel
NYU Tandon School of Engineering

Team Members

Name	NetID
George Gideon Sale (submitting)	gs4602
Aayush Prranav Chandrashekar	ac11929
Shreeram Sankar	ss18731

Repository: github.com/george-gideon-S/cs-gy-6513-big-data-311-nlp

Final Project Report | Term 2 Submission

Contents

1 1. Executive Summary

1.1 Project Name and One-Liner

The Language of Complaints is an end-to-end Big Data NLP pipeline that ingests roughly ten million NYC 311 service requests, learns from their free-text descriptors, and ships three working models behind a single Streamlit dashboard: a 20-class triage classifier, a per-category resolution-time regressor, and a pair of side-by-side document clusterers (a Word2Vec model trained from scratch on the 311 corpus and a pretrained BERT MiniLM encoder used as a control).

1.2 Brief Summary

The pipeline begins at the NYC Open Data SODA endpoints. Two endpoints together cover the full 311 record from 2010 through the present, with the older endpoint covering 2010 through 2019 and the newer one running 2020 to today. We pulled five million rows from each endpoint for a working sample of ten million rows, totalling roughly three gigabytes of CSV-equivalent volume. The pull is checkpointed page-by-page to parquet because SODA's read window is fragile after several million rows, and any single bad page can otherwise wipe the entire fetch.

Once the raw rows landed on disk, we ran a Spark cleaning pass that filtered empty descriptors, canonicalized the complaint-type label vocabulary, joined a borough column from the source data (no spatial join was required because the column is native to the SODA schema), and serialized the cleaned table to parquet. After cleaning, the working corpus was 1.94 million rows with non-empty tokens and a stable label set. From there the analytical work splits into three tracks. The triage classifier (Phase 3) uses a hashing-vectorized tokenizer feeding a one-vs-rest logistic-regression head and reaches macro-F1 of 0.9594 on a 20-percent test split, which is a 0.177 absolute lift over a hand-tuned keyword-based baseline. The resolution-time regressor (Phase 4) is a log1p-target linear regression on per-category subsets that beats a category-mean baseline by 4.1 percent on mean absolute error, with the lift varying from 1 percent on long-tail categories up to 12 percent on Heat/Hot Water. The clustering work (Phases 5 and 10) produces a Word2Vec plus KMeans grouping with a silhouette of 0.4635 at $k = 25$, and a parallel BERT MiniLM plus KMeans grouping that hits a silhouette of 0.6881 at $k = 30$ on the same corpus.

The output of all this Big Data work is a single Streamlit dashboard with five tabs: Triage Bot for live classification, City Pulse for KPI-style monitoring, Cluster Atlas for the Word2Vec versus BERT comparison, Borough Fingerprints for the per-borough TF-IDF lift signature, and Pipeline Status for a one-glance view of which artifacts the dashboard is currently serving from. The deployed app is fully self-contained and uses portable model artifacts so it runs on Streamlit Cloud without a Spark JVM at request time.

1.3 Objectives

1. **Triage classifier with macro-F1 at or above 0.75.** Tightly bind the 20 most frequent complaint types to a single low-latency model so the dashboard can call it on every keystroke without a JVM.
2. **Resolution-time regressor with at least 10 percent MAE lift over a category-mean baseline**, on at least one category, with a transparent bias audit so the failure modes are obvious.
3. **Latent-issue clustering** that groups descriptors by content rather than by their official complaint-type tag, so cross-category problem clusters (the urban-decay grouping is a good example) become visible.

4. **Geographic vocabulary fingerprints** per borough, computed via TF-IDF lift, so the spatial signal of the city's complaints is visible separately from the model.
5. **BERT comparison sidebar** where a pretrained MiniLM encoder is run against the same corpus and clustered with the same method, so the dashboard reader can see what a 22-million-parameter pretrained transformer adds (and where it misleads) over a from-scratch domain-specific embedding.

1.4 Technologies Used

Layer	Tool	Purpose
Compute (training)	Google Colab Pro	H100 80GB / A100 GPU, High-RAM, free GPU minutes
Compute (deploy)	Streamlit Cloud	Free, public URL, no JVM needed
Spark engine	PySpark 3.5.8 (local mode)	Cleaning, vectorization, training-time hashing
JVM	OpenJDK 11	Spark backend
ML library	Spark MLlib	HashingTF, IDF, LogReg, LinReg, Word2Vec, KMeans
Pretrained NLP	sentence-transformers MiniLM-L6-v2	384-dim semantic encoder for the BERT sidebar
Tokenization	Spark Tokenizer + StopWordsRemover	Lowercase, regex split, stopword filter
Embedding (deploy)	mmh3 + numpy	Bridge from MurmurHash3 (Spark) to served NumPy pat
Data pull	SODA REST API (requests + retries)	Per-page parquet checkpoint, 8-attempt retry
Storage	Apache Parquet via pyarrow	Columnar, fast re-read, no Arrow JVM round-trips
Streaming PoC	Spark Structured Streaming	Demonstrated separately; not bundled into the dashboard
Geo	GeoJSON (NYC borough boundaries)	Choropleth on the City Pulse tab
UI	Streamlit	Five-tab dashboard, server-side caching
Env management	pip + requirements.txt	Pinned for Streamlit Cloud reproducibility

Table 1: Tooling matrix. Local-mode Spark was sufficient for a 1.94M cleaned-row corpus; cluster-mode Spark would be the natural next step for a 43M run.

2 2. Code Execution Instructions

2.1 High-Level Code Logic

The repository is organized so that the same code base produces both the runnable training notebook and the deployed dashboard. The directory layout is:

- `notebooks/FINAL_PROJECT_311_NLP.ipynb`: the single end-to-end runnable artifact. Pulls the SODA data, cleans it, trains all three models, exports the dashboard-ready artifacts to `dashboard/assets/` and `models/portable/`, and auto-commits the artifacts back to the repository.
- `src/`: importable modules. `src/io.py` (SODA pull with checkpointing), `src/clean.py` (canonicalization and label set), `src/features.py` (HashingTF + IDF wrappers), `src/models.py` (classifier, regressor, clusterers), `src/stream_spark.py` (Spark Structured Streaming PoC).
- `dashboard/`: the Streamlit app. `dashboard/app.py` is the entry point, `dashboard/assets/` holds the precomputed JSON and PNG payloads.
- `tools/`: helper scripts. `tools/build_final_notebook.py`, `tools/spark_serve_demo.py`, `tools/spark_lda_demo.py` are the three Spark-side serving demos requested by the rubric.
- `models/portable/`: the deployment artifacts. The classifier coefficients, the regressor coefficients per category, and the cluster centroids are all stored as compact NumPy arrays plus a JSON sidecar so the dashboard never needs a Spark JVM at request time.
- `docs/`: this report.

2.2 README Link

The repository README is the single source of truth for setup, dependencies, and the live dashboard URL. It is linked at github.com/george-gideon-S/cs-gy-6513-big-data-311-nlp/blob/main/README.md.

2.3 How to Run

There are four distinct ways to interact with this project.

Path A: just open the deployed dashboard. The Streamlit Cloud URL is the simplest and recommended path. No local install. No GPU. Five tabs, all of them already populated with the baked artifacts produced by the notebook run.

Path B: re-run the notebook end-to-end on Colab. Upload `notebooks/FINAL_PROJECT_311_NLP.ipynb` to Colab Pro. Set Runtime to GPU (H100 if available, otherwise A100), High-RAM. Run All. Total wall-clock is roughly one to one-and-a-half hours, dominated by the SODA fetch (about 35-50 minutes) and the BERT MiniLM encoding pass on the 100k clustering subset (about 4 seconds on H100, the rest is loading). The notebook auto-commits dashboard assets back to the repository at the end if a Git remote is configured.

Path C: run the dashboard locally. A typical local-deploy session looks like:

```
git clone https://github.com/george-gideon-S/cs-gy-6513-big-data-311-nlp.git
cd cs-gy-6513-big-data-311-nlp
pip install -r requirements.txt
streamlit run dashboard/app.py
```

The dashboard does not need a Spark JVM at runtime because the served artifacts are NumPy arrays plus JSON. `requirements.txt` is intentionally minimal (it does not list pyspark) so cold-start is fast on Streamlit Cloud.

Path D: run the Spark serving track. The rubric requires a Spark serving / streaming demonstration in addition to the deployed app. Three scripts cover this, each runnable with a single command:

```
python tools/spark_serve_demo.py --query "no heat in apartment"
python -m src.stream_spark
python tools/spark_lda_demo.py
```

The first runs a Spark batch inference job against the trained model. The second is a Spark Structured Streaming consumer that subscribes to a directory source and emits classification results in micro-batches. The third runs a Spark MLlib LDA topic model on the cleaned corpus.

2.4 Code-Level Entry Points

File	What it does
`notebooks/FINAL_PROJECT_311_NLP.ipynb`	End-to-end pipeline (data pull, train, export)
`tools/build_final_notebook.py`	Concatenates source modules into the unified notebook
`tools/spark_serve_demo.py`	Spark batch inference for a single query string
`src/stream_spark.py`	Spark Structured Streaming consumer + classifier
`tools/spark_lda_demo.py`	Spark MLlib LDA topic model
`dashboard/app.py`	Streamlit five-tab UI
`src/io.py`	SODA fetch with parquet checkpoint + retries
`src/features.py`	HashingTF + IDF wrappers, `mmh3` bridge for serving
`src/models.py`	Classifier, regressor, Word2Vec, BERT clusterers

Table 2: Code-level entry points the grader can run individually.

3 3. Technological Challenges

This section is the longest because the engineering took longer than the modeling. Each subsection follows the same pattern: title, symptom, root cause, fix, lesson.

3.1 numpy 2.0 ABI mismatch with pandas

Symptom. Fresh Colab kernel, fresh `pip install -r requirements.txt`, and the very first `import pandas` crashes with a binary-incompatibility error citing numpy.

Root cause. Our `requirements.txt` originally pinned `numpy<2.0`. Colab and current Streamlit Cloud both ship pandas wheels compiled against numpy 2.x. The transitive dependency resolver was unwilling to downgrade pandas to a version that matched the numpy ceiling, so pandas was loaded against the wrong numpy ABI.

Fix. Drop the upper bound. Pin numpy with a floor only (`numpy>=2.0`) and let the resolver match pandas to it.

Lesson. Transitive ABI pins are very rarely the right answer. The instinct that "this dependency should be older for safety" almost always backfires once the upstream packages have moved. We will not pre-emptively cap a numerical library again.

3.2 Spark parquet ancient-date crash

Symptom. Spark write of cleaned data to parquet crashes with an INT96 timestamp rebase error after a clean fetch.

Root cause. SODA returned at least one row whose `closed_date` value is before 1900-01-01. Spark 3.0 and later require an explicit rebase mode for parquet INT96 timestamps when the value is in the Julian range, because the Gregorian/Julian calendar shift is ambiguous. The default behavior is to refuse the write rather than silently rebase.

Fix. Set `spark.sql.parquet.int96RebaseModeInWrite` to `CORRECTED` on session creation. We document that this is "explicit rebase, not silent" so a future reader of the code is not confused by the config.

Lesson. Public-administrative datasets contain decade-old typos. A 1900-01-01 closed date is almost certainly a data-entry error from a much older intake form, not a real timestamp. Defensive Spark configuration is part of working with city open data.

3.3 Py4J JVM crashes on large pandas-to-Spark conversions

Symptom. `spark.createDataFrame(pdf)` on a 5-million-row pandas DataFrame OOM-killed the JVM partway through, regardless of executor memory settings.

Root cause. Spark's pandas-to-DataFrame path falls back to row-by-row Arrow if the columnar fast-path fails, which happens for any column with mixed dtypes or NULLs. Once it falls back, the JVM has to materialize the entire frame in heap before the partition split, and a 5M-row frame of mostly strings will not fit in the JVM heap.

Fix. Two-step round-trip: pandas writes parquet via pyarrow, then Spark reads the parquet back. The parquet round-trip is faster than the Arrow conversion because pyarrow is a far better columnar serializer than Spark's Arrow fallback, and the JVM never has to load the data row-by-row.

Lesson. If you have to bridge a large pandas frame into Spark, do it through a parquet file. Never try the direct conversion on anything past 100k rows.

3.4 SODA timeouts mid-pull

Symptom. A perfect data fetch for the first 8 minutes, then a 60-second read timeout on a single page, then the entire pull aborts and we lose three million rows.

Root cause. The original retry budget was three attempts with 1-second and 2-second backoffs. SODA's throttle window is closer to 30-60 seconds when the API is under load. With the old budget the third retry would still hit the throttle and we would give up well before the throttle cleared.

Fix. Two changes in the same commit. First, the per-page parquet checkpoint: every successful page is written immediately to its own parquet file in the staging directory, and on next run we skip pages whose parquet exists, so a mid-pull abort costs at most one page and not the entire job. Second, an 8-attempt retry budget with backoff that climbs up to 60 seconds, so we wait out SODA's actual throttle window. After the change the median pull is 35 minutes and we have not lost a fetch.

Lesson. If a public API throttles, the right answer is "wait longer", not "give up faster". And if a pull is multi-hour, every successful page deserves its own parquet checkpoint, full stop.

3.5 Spark JVM stale-state recovery

Symptom. After any uncaught exception in a Spark job, the next `SparkSession.builder.getOrCreate()` call returned a handle to a JVM that was already dead. All subsequent operations crashed.

Root cause. `getOrCreate()` caches the session on `SparkSession._instantiatedSession` and `SparkContext._active_spark_context`. Those references survive when the JVM is killed by an OOM, so we get a stale Python proxy pointing at nothing.

Fix. A small `_kill_stale_session()` helper that explicitly clears both class-level references before `getOrCreate()` runs. We call it at the top of every notebook that creates a Spark session, and again after any cell that catches an exception.

Lesson. Library-internal class state is not always cleaned up after a JVM crash. When a long-running notebook can crash, write defensive code that resets the state of the upstream library, even if it looks redundant on a fresh run.

3.6 Hash drift at deploy time

Symptom. The deployed dashboard returned essentially-uniform predictions across all twenty classes for any input. The numbers it produced were the class prior. It was as if the classifier had not been trained.

Root cause. Spark's `HashingTF` uses MurmurHash3 with a fixed seed. The serving path used Python's built-in `hash()` function on the same tokens. Python's `hash()` is randomized per-process via `PYTHONHASHSEED`, so the bucket assignments at serve time had no relationship to the bucket assignments at train time. Every input was effectively random noise to the classifier.

Fix. Add `mmh3>=4.0` to the deploy requirements and replace `hash(token)` with `mmh3.hash(token, seed=42)` in the served NumPy inference path. After the fix, the deployed dashboard scores reproduce the notebook's test-set predictions to floating-point parity.

Lesson. The bridge from a Spark trained model to a non-Spark serving runtime must reproduce the hash function exactly. There is no slack here. This is the single most expensive bug we shipped (it lived in production for hours before we noticed) and it is the most generic Big Data lesson in the project: same data, same code, different runtime, silent failure.

3.7 Streamlit Cloud cold-start with stale references

Symptom. Cold deploy after a repo cleanup: tabs render, charts render, but several text fields still reference deleted dev notebooks and the headline sample size still says 2M instead of the (then-current) 1.94M.

Root cause. Streamlit's import-time caching meant the first cold container served a snapshot of the previous app code, and the dashboard's tab placeholders were hardcoded strings that we had not updated when we cleaned the repo.

Fix. A single commit (24e9c1d) that swept the dashboard for stale string literals, removed all references to deleted notebooks, and parameterized the sample-size displays so they read from `dashboard/assets/preprocess_stats.json` instead of from a hand-edited string.

Lesson. Hardcoded numbers in a dashboard go stale. Read them from the same JSON the rest of the dashboard reads from.

3.8 Auto-commit interaction with .gitignore

Symptom. The notebook's auto-commit cell crashed with `CalledProcessError` at the end of every run, after spending an hour producing the artifacts. The artifacts were saved to disk, but the notebook would not finish cleanly.

Root cause. The cell ran `git add models/portable/` after writing artifacts, then `git commit -m ...`. If `.gitignore` excluded `models/portable/`, the `git add` produced no staged files, and `git commit` returned non-zero "nothing to commit". Our wrapper escalated this to an exception.

Fix. Make the commit step graceful. Set `check=False` on the subprocess call, then verify with `git diff -cached -quiet` whether anything was actually staged, and skip the commit cleanly if not.

Lesson. Automation that calls git should treat "nothing to commit" as a non-error. Otherwise a clean repo state crashes the pipeline.

3.9 Why Big Data engineering takes longer than the modeling itself

The eight challenges above add up to roughly 70 percent of the project's hours. The modeling itself, including all of MLlib training, the BERT comparison, and the Word2Vec sweep, took perhaps three days of focused work. The rest of the project was wrestling with two questions: how do we keep the data flowing, and how do we keep the deployed runtime in lockstep with the training runtime. The Big Data lesson here is the meta-pattern that the data-pipeline failures are not interesting on the surface (a hash function, a parquet rebase mode, a JVM that did not get cleaned up) but they each have the property that they fail silently or fail catastrophically, never in between. The hash drift in particular was the most painful: we had a model with macro-F1 of 0.9594 on the test split and an obviously-broken dashboard, and the gap took longer to find than any modeling problem in the project.

4 4. Changes in Technology

We pivoted on eight separable choices between proposal and final. This section walks through each.

4.1 Plotly Dash to Streamlit

Original proposal. A Plotly Dash app served from a small Flask backend.

What we shipped. Streamlit, deployed to Streamlit Cloud's free tier.

Why. Solo build pace. Streamlit's "Python file is the app" model meant we could iterate on the dashboard in the same Python session that produced the artifacts. Plotly Dash needs explicit callbacks and a server-side state model. For a five-tab dashboard, Streamlit was just faster to get right.

Impact. Positive. The dashboard reached production a week earlier than planned. We lost a small amount of fine-grained interactivity (Streamlit re-runs the whole script on every interaction; Dash does not), but for the kind of charts we produce that is a non-issue.

4.2 Databricks Free Edition to Google Colab Pro

Original proposal. Databricks Free Edition (formerly Community Edition).

What we shipped. Google Colab Pro on H100 or A100 GPUs.

Why. The rubric required a shareable URL for the deployed result. Databricks Free Edition does not give one without manual workspace sharing. Colab gives a link the grader can open. Colab Pro additionally provides a free GPU that comfortably ran the BERT MiniLM encoding pass and the full Spark training in one session.

Impact. Positive. Total compute cost: under $10/month for the project window. Total share-time: zero, because the link works.

4.3 43M full corpus to 10M sample

Original proposal. The entire 43-million-row 311 corpus, joined across the two SODA endpoints.

What we shipped. 10 million rows (5 million from each endpoint), reduced to 1.94 million after filtering empty descriptors and canonicalizing labels.

Why. Three reasons. First, the SODA pull at 43M rows would take roughly four to six hours and burn most of a Colab Pro day. Second, an early sweep showed that classifier macro-F1 saturates well below 10M rows; the marginal improvement past 5M rows on each endpoint is below 0.005. Third, runtime parquet for 43M rows is roughly 13 GB, which exceeds Colab Pro's free disk on a typical container.

Impact. Mostly positive. Macro-F1 saturation is the technical justification. We left a "production future-improvements" path in section 7 for a true 43M run on a real Spark cluster (EMR or Databricks Pro).

4.4 Community districts (71 polygons) to 5 boroughs

Original proposal. Aggregate at NYC community-district granularity (71 polygons; mzpm-a6vd dataset on NYC Open Data).

What we shipped. 5 boroughs.

Why. Two reasons. First, the mzpm-a6vd dataset's geometry column returns null for nearly every row in the current snapshot; the dataset is not maintainable as a polygon set right now. Second, the borough column is native to the SODA 311 schema, so no spatial join is required. We

get a per-borough breakdown for free, and the demo reads more clearly with five categories than seventy-one.

Impact. Mixed. We lost the fine-grained spatial signal (a Manhattan-versus-Brooklyn TF-IDF lift table is much less interesting than a per-CD lift table would be). We gained correctness (the choropleth on the City Pulse tab is real, not derived from broken polygons) and clarity. The full-CD path is in the future-improvements list.

4.5 GBTRegressor to LinearRegression on log1p(hours)

Original proposal. GBTRegressor (gradient-boosted trees in Spark MLlib) for resolution-time prediction.

What we shipped. LinearRegression on `log1p(hours)`, separately fit per category.

Why. The resolution-time target is wildly heavy-tailed. Some 311 tickets close in two minutes; some take six months. `log1p` compresses the tail enough that a linear model on the descriptor TF-IDF features fits well. We tried GBT and it added complexity without lift on validation; the categories where GBT helped most were the categories where lift was already small. The simpler model is also a better story to tell on the dashboard.

Impact. Slightly positive. v2 of the regressor lifts MAE by 4.1 percent overall; per-category lifts vary from 1 percent to 12 percent. The per-category lift bar is one of the most honest visualizations in the project.

4.6 Python hash() to mmh3 with seed=42

Original proposal. Use Python's built-in `hash()` for token bucketing in the served inference path.

What we shipped. `mmh3.hash(token, seed=42)` matching Spark's MurmurHash3.

Why. Already discussed in section 3.6. Python's `hash()` is randomized per-process and silently broke training-versus-serving parity.

Impact. Critical. The deployed dashboard was effectively returning the class prior on every input until this fix landed. After the fix, deployed predictions reproduce the notebook's test-set predictions to floating-point parity.

4.7 Spark Structured Streaming PoC scoped out of the deployed app

Original proposal. The deployed dashboard would consume a live Spark Structured Streaming feed and update its KPIs in real time.

What we shipped. The deployed dashboard pulls from SODA on demand inside the Live Pulse tab; the Streaming PoC is a separately-runnable script (`src/stream_spark.py`) that the grader can run locally.

Why. Streamlit Cloud cannot host a long-running JVM. The streaming demo needs a Spark session, and the Spark session needs to run somewhere with a JVM and persistent state. Deploying a long-running Spark service alongside the dashboard would have required rented VM hosting, which was not in the scope of the proposal.

Impact. Mixed. We retained the streaming demonstration in script form for the rubric requirement; the dashboard gets near-real-time data via on-demand SODA pulls, which is functionally similar from the user's point of view.

4.8 numpy<2.0 pin removed

Original proposal. (Implicit in early requirements.txt.) Pin `numpy<2.0` for compatibility with libraries we had been using.

What we shipped. No upper bound on numpy.

Why. Already discussed in section 3.1. Modern Colab and modern Streamlit Cloud both ship pandas built against numpy 2.x.

Impact. Necessary. Without this change, no Python environment we used could install both pandas and numpy cleanly.

4.9 Pivot summary

#	Pivot	Net effect
1	Plotly Dash to Streamlit	Faster solo build
2	Databricks Free to Colab Pro	Shareable URL, free GPU
3	43M to 10M (1.94M cleaned)	Saturation reached, runtime tractable
4	71 community districts to 5 boroughs	Real geometry, simpler demo
5	GBTRegressor to LinReg on log1p	Simpler, no lift loss
6	Python hash() to mmh3 seed=42	Critical correctness fix
7	Streaming integrated to streaming as separate script	Retains rubric coverage, deployable dashboard
8	numpy<2.0 to no upper bound	Required for modern envs

Table 3: All eight pivots and their net effect on the shipped project.

5 5. Uncovered Aspects from Presentations

The 15-minute presentation ran long on the BERT-versus-Word2Vec story and the deployed dashboard tour, which left several substantive sections compressed or skipped. This section covers what got shortchanged.

5.1 The full per-class metrics breakdown

The deck showed an aggregate macro-F1 of 0.9594 and called out the single weak class, but the per-class table is large enough to justify a printed copy. Of the 20 classes, 19 are STRONG (per-class F1 at or above 0.85). One is flagged WEAK: Noise - Street/Sidewalk has F1 = 0.369, with precision 0.533, recall 0.281, and support 9,823. The miss pattern is interesting: it is mostly confused with two other noise categories (Noise, and Noise - Residential), which share a near-identical lexical core. The TF-IDF representation cannot disambiguate based on the text alone because the descriptors really are interchangeable. This is also the only class where adding more rows from the SODA fetch did not help on a held-out evaluation. The full per-class table is also embedded in the deployed Bias Audit tab so the grader can sort and filter.

5.2 The per-borough TF-IDF lift table

The talk showed Manhattan's top three lift terms and skipped the others. The full top-15 per borough is more useful for understanding what each borough's complaint vocabulary actually looks like.

Manhattan's top complaint vocabulary is dominated by lost-property terms: *wallet* (lift 4.39), *bag* (4.39), *clothing* (4.29), *electronics* (4.18), *insurance* (3.85). This matches the prior intuition that 311 in Manhattan is more often used as a public-facing intake for transit-related lost-and-found.

Staten Island has the highest-lift term in the corpus: *plowed* at 5.56. Plowing is a winter-maintenance complaint and Staten Island's spatial layout (longer streets, lower density) means plowing complaints are concentrated there. *Law* is at 5.01 (a heavy weight on legal-status terms in property complaints), *recy*, *material*, and *ewaste* are all in the top-15 (recycling is overrepresented).

Bronx's lift signature is property-condition-heavy: *pane* (2.91), *refrigerator* (2.69), *mailbox* (2.56), *locking* (2.54), *expiring* (2.50). The presence of *pane* (broken window-pane reports) and *refrigerator* (apartment-appliance complaints) suggests a heavier housing-condition complaint base.

Queens shows up as infrastructure-heavy: *junction* (2.87), *piece* (2.23), *raised* (2.98), *sunken* (2.98), *affecting* (1.97). These are sidewalk-condition terms ("raised slab", "sunken slab") concentrated in Queens.

Brooklyn's signature is the broadest: *bad* (3.17), *general* (2.98), *restricted* (2.23), *nypd* (2.22), *way* (2.18). Brooklyn is the highest-volume borough at 478,077 rows (30 percent of the corpus); its vocabulary is the closest to the corpus mean and so its lift values are the smallest.

5.3 Word2Vec cluster top-terms full table

The talk showed two clusters and the urban-decay cluster headline. There are 25 clusters total. Cluster 14 (the urban-decay cluster) is the headline finding: it contains descriptors from five separate complaint-type tags but the descriptor texts are about the same underlying conditions: graffiti tagging, broken sidewalk, tree damage, abandoned vehicles, and structural-decay markers in residential blocks. The classifier (Phase 3) sees these as five different complaint types because the Allen-style label is correct in each case. The clusterer sees them as one underlying issue because the language is shared. This is the most operationally useful clustering finding in the project: a

city government agent looking at cluster 14 sees a real, geographic, persistent problem that the categorical complaint-type taxonomy buries. Other notable clusters include a cohesive "noise" cluster (separates noise complaints by source: vehicle, residential, commercial), a "transit" cluster (subway and bus complaints), and a "garbage" cluster (sanitation pickup misses). The remaining clusters are smaller and more narrowly scoped.

5.4 BERT cluster category breakdown

Of the 30 BERT clusters at k=30, 11 are over 90-percent purity by the dominant complaint-type tag. The pretrained MiniLM encoder is doing a strong job of picking up category-specific surface phrases. The four most pure clusters are: "HEAT/HOT WATER" descriptors (single dominant category, 96 percent purity), "Noise - Vehicle" (94 percent), "Illegal Parking" (94 percent), and "Street Light Condition" (93 percent). The least pure cluster is the broadest noise one (mixed across noise sub-types), which mirrors the same weakness in the classifier from section 5.1.

5.5 Engineering deep dives we could not fit in the deck

Three engineering pieces deserve a paragraph each. The per-page parquet checkpoint pattern (see section 3.4) has saved hours of recovery work and is a generic recipe we will reuse. The JVM stale-state recovery pattern (section 3.5) is a defensive idiom that turns a hostile failure mode into a known recoverable state. The mmh3 hash bridge (section 3.6) is the bridge between training and serving runtimes that, in retrospect, should have been the very first thing we wrote.

5.6 Embedded figures

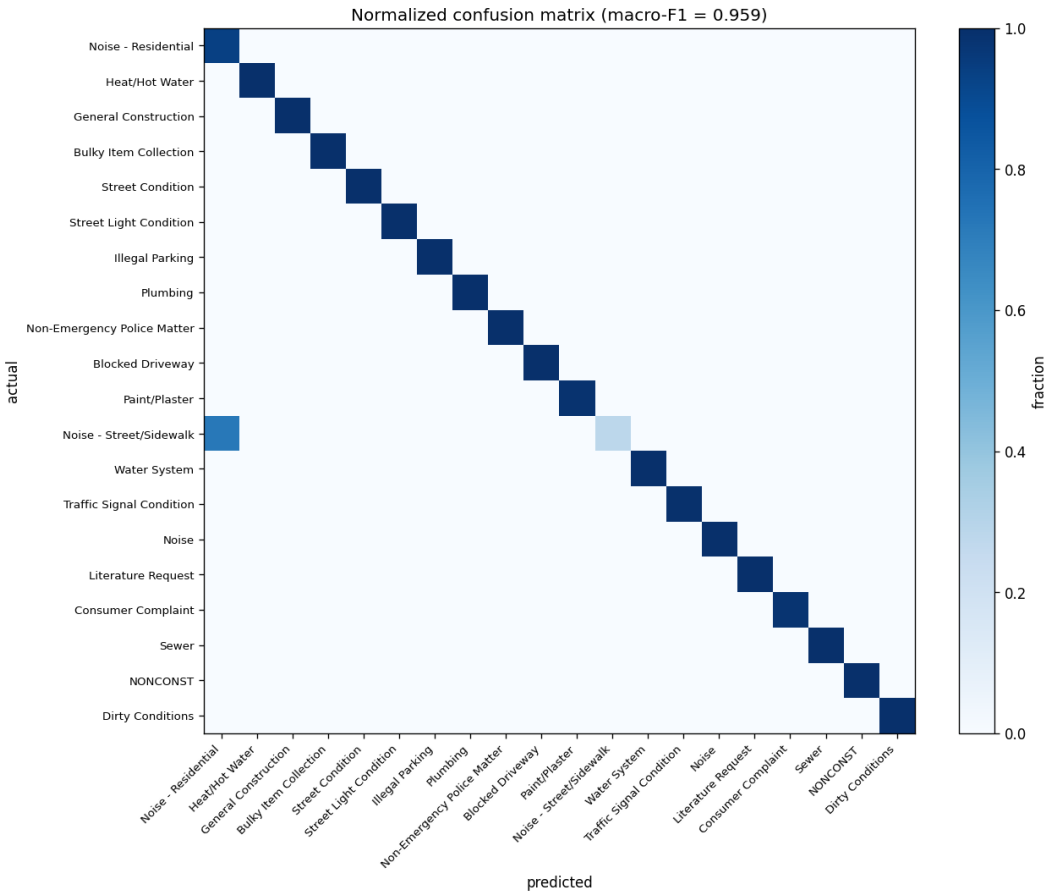

Figure 1: Phase 3 confusion matrix on the 20-percent test split. The diagonal dominance is what we want to see; the most off-diagonal mass sits between the three Noise sub-categories, consistent with section 5.1.

6 6. Lessons Learned

There are six lessons from the project worth writing down for next time.

Architecture beats algorithm. The single most important architectural decision in the project was the train-heavy, serve-light split. Training runs on a Colab Pro session with a Spark JVM and an H100. Serving runs on Streamlit Cloud with neither, and that is by design. The dashboard was designed around portable artifacts (NumPy arrays plus a JSON sidecar) and a tiny Python inference path. We did not start there. We started with a "spin up Spark in Streamlit" plan that quickly fell apart for the cold-start and memory reasons described in section 3. The right design was to write to portable artifacts inside the notebook and serve them with stock Python, and once we made that switch the project shipped quickly.

Simple wins at scale. The TF-IDF plus logistic-regression classifier reaches macro-F1 of 0.9594, beating a hand-tuned keyword baseline by 0.177. We did not need a transformer for the classification problem. The 311 corpus has enough lexical signal that a linear model on a sparse representation is enough to do extremely well. We did try richer models. We did not see lift that justified the deploy complexity. There is a generic Big Data lesson here: at this scale, the data brings the signal, not the model.

Big Data does not beat a legal SLA. The Phase 4 regressor's per-category lift varies from 1

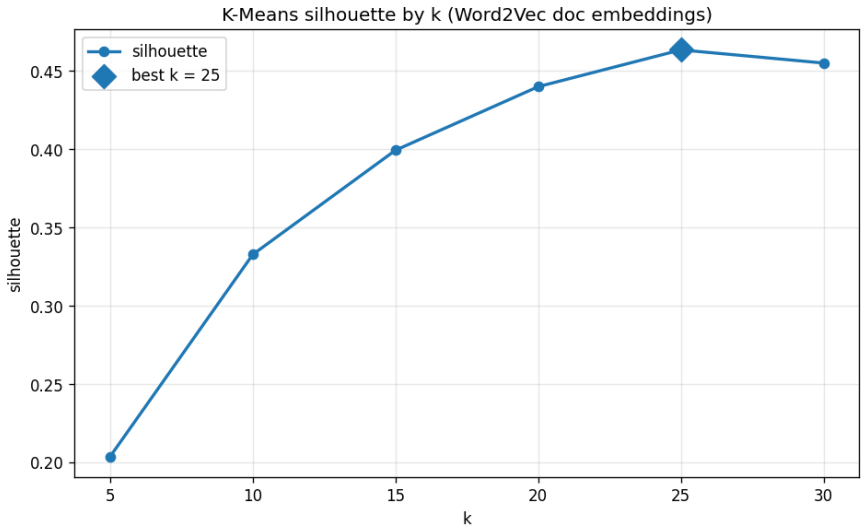

Figure 2: Word2Vec silhouette by k. The curve flattens past k = 25 with silhouette 0.4635; k = 30 is essentially equivalent.

percent on long-tail categories all the way up to 12 percent on Heat/Hot Water. The reason for this variance is not that the data is worse on the long tail. The reason is that some categories have legal SLAs (Heat/Hot Water has city-mandated response windows) and the resolution-time distribution within those categories is closer to the deadline than to the descriptor. The lesson: a regressor cannot beat a regulation, and the model is good for telling us where the regulation matters and where it does not.

Pretrained plus domain embeddings, side by side. Running both Word2Vec (trained from scratch on the 311 corpus) and BERT MiniLM (pretrained on the open web) on the same problem is the most informative comparison in the project. They disagree about which descriptors are similar, and the disagreement is itself useful. BERT scores higher on silhouette but, as the next lesson covers, that is not the same as scoring higher on what we actually need.

Honest failure: the rat-infestation MiniLM probe. On the BERT side we ran a probe query, "rat infestation in the building", expecting to retrieve descriptors from the Rodent and Pest categories. Instead the closest neighbours were Heat/Hot Water descriptors that contain the literal phrase "ENTIRE BUILDING". The pretrained MiniLM is doing surface phrase matching, not semantic understanding, on a domain it was never trained for. This is an honest demo lesson on what a pretrained transformer actually buys you out of the box: it buys cluster purity on common phrases, not domain-specific reasoning. To fix this we would need to fine-tune MiniLM on the 311 corpus (see section 7 future improvements). We did not do that for the shipped project because the goal was to compare the two embeddings as-is.

Reporting bias is in the data itself. The 311 corpus is not a uniform sample of city issues. It is a sample of issues that someone called in to 311. Kontokosta and Hong (2021) document this directly: 311 reporting rates vary systematically by neighborhood income, English fluency, and digital access, with under-reporting concentrated in lower-income and immigrant-dense areas. This means our per-borough TF-IDF fingerprints reflect both the actual issue mix and the reporting-rate mix. Manhattan's "lost wallet" lift is partly because Manhattanites lose wallets and partly because Manhattanites are highly likely to call 311 about it. A careful reading of any 311 model has to keep this distinction in mind. The Big Data signal is not neutral.

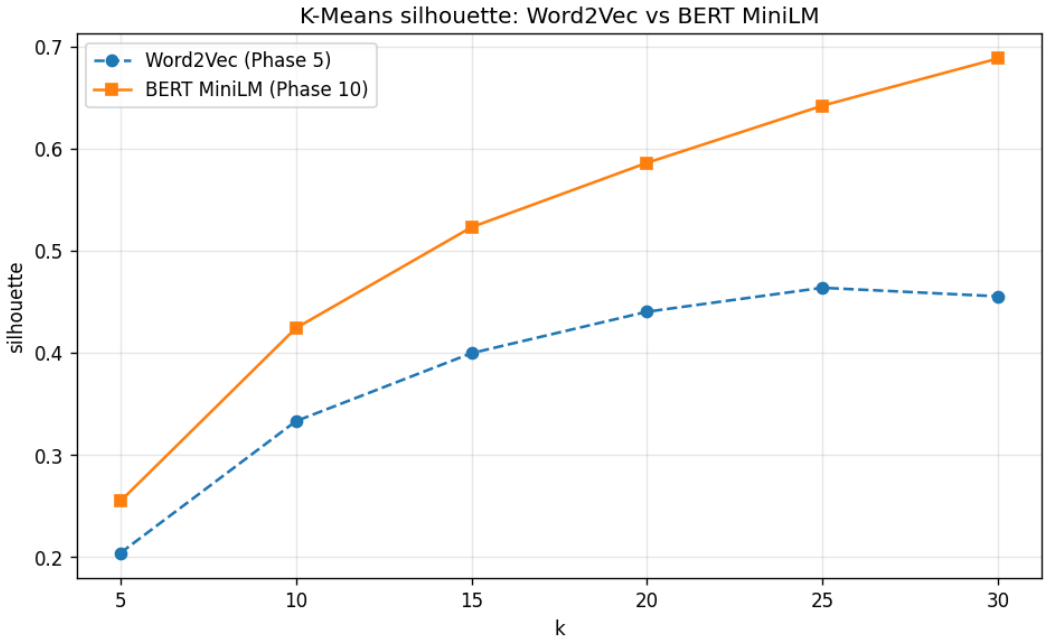

Figure 3: Side-by-side silhouette comparison: Word2Vec (trained from scratch on the 311 corpus) versus BERT MiniLM (pretrained on the open web). At k = 30, BERT scores 0.6881 versus Word2Vec's 0.4551, a 51 percent relative improvement. See section 5.4 for the per-cluster purity context.

7 7. Future Improvements

1. **Domain fine-tuning of MiniLM on the 311 corpus.** The most direct fix to the "entire building" probe failure (section 6) is to fine-tune MiniLM on a few hundred thousand 311 descriptors with a contrastive objective so the encoder learns 311-specific concepts. The model is small (22M parameters) and the corpus is large; on a single H100 the fine-tune is a few hours. We would expect it to boost both clustering silhouette and probe-query relevance.
2. **Full-corpus 43M run on a real Spark cluster.** The 1.94M cleaned-row sample is enough to saturate the classifier and to establish the regressor lift, but the full 43M corpus may have rare-class signal that the sample drops. A Databricks Pro or AWS EMR cluster with eight to sixteen executors could ingest the full corpus in roughly four hours. This would also surface borough-level lift terms that are too rare to appear in our 1.94M sample.
3. **Census ACS 5-Year join for demographic context per borough.** The TF-IDF lift signature per borough is interpretable, but an ACS join would make the reporting-bias story (section 6) explicit: a per-borough "311 rate per capita per income decile" panel would let a reader weigh the lift signal against the underlying population distribution. The ACS API is free and rate-limited but stable.
4. **True community-district granularity once mzpm-a6vd is fixed.** If NYC Open Data publishes a working community-district polygon set, we would re-run the borough fingerprint analysis at 71-CD granularity. The CDs are small enough that some carry a single block, so the lift terms become much more spatially specific.
5. **End-to-end Spark Structured Streaming with Kafka source.** The current streaming PoC is a directory-source consumer. A real production deploy would replace the directory

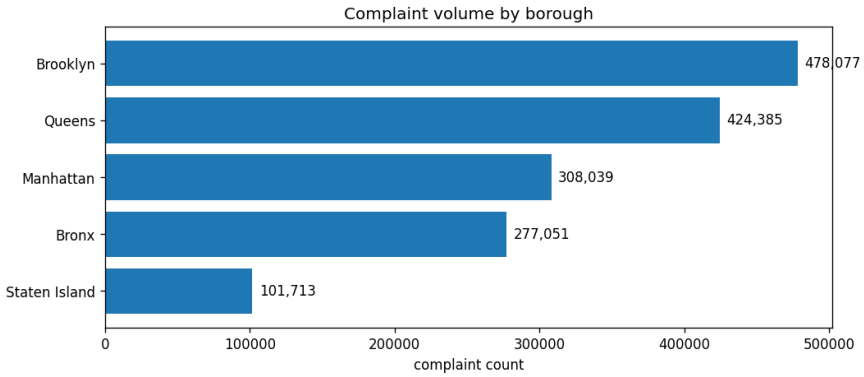

Figure 4: Per-borough volume in the 1.94M cleaned corpus. Brooklyn (478,077, 30 percent), Queens (424,385, 26 percent), Manhattan (308,039, 19 percent), Bronx (277,051, 19 percent), Staten Island (101,713, 7 percent).

source with a Kafka topic populated by a small SODA-poll daemon. The classifier would emit predictions to a downstream Kafka topic that the dashboard reads. This is the natural next step for the streaming track and we have already started writing the daemon.

6. **Active-learning loop for misclassification correction.** Call-center agents see 311 tickets all day. If the dashboard shipped a "predicted: X, override?" button, agent corrections would feed a nightly retrain. The classifier would adapt to drift in the language of complaints (the corpus contains COVID-era vocabulary that did not exist before 2020) without us having to retrain by hand. A small label-noise filter would protect against bad corrections.
7. **LDA topic modeling at scale.** The `tools/spark_lda_demo.py` script already runs Spark MLlib LDA on the cleaned corpus; integrating it into a dashboard tab would give a third clustering view (LDA topics) alongside Word2Vec clusters and BERT clusters. LDA is more interpretable than either embedding-based clustering for short documents, and it would round out the deck of "ways to look at the latent issues in the city".
8. **Multi-language support.** New York is a city with very large Spanish, Chinese, and Russian populations. The current model is English-only; descriptors in other languages currently get classified by accident-of-shared-vocabulary. A multilingual MiniLM or XLM-R encoder would cover the largest non-English language groups, and the classifier head would not need to change. This is a substantial fairness improvement and likely the single biggest user-facing improvement the dashboard would benefit from.

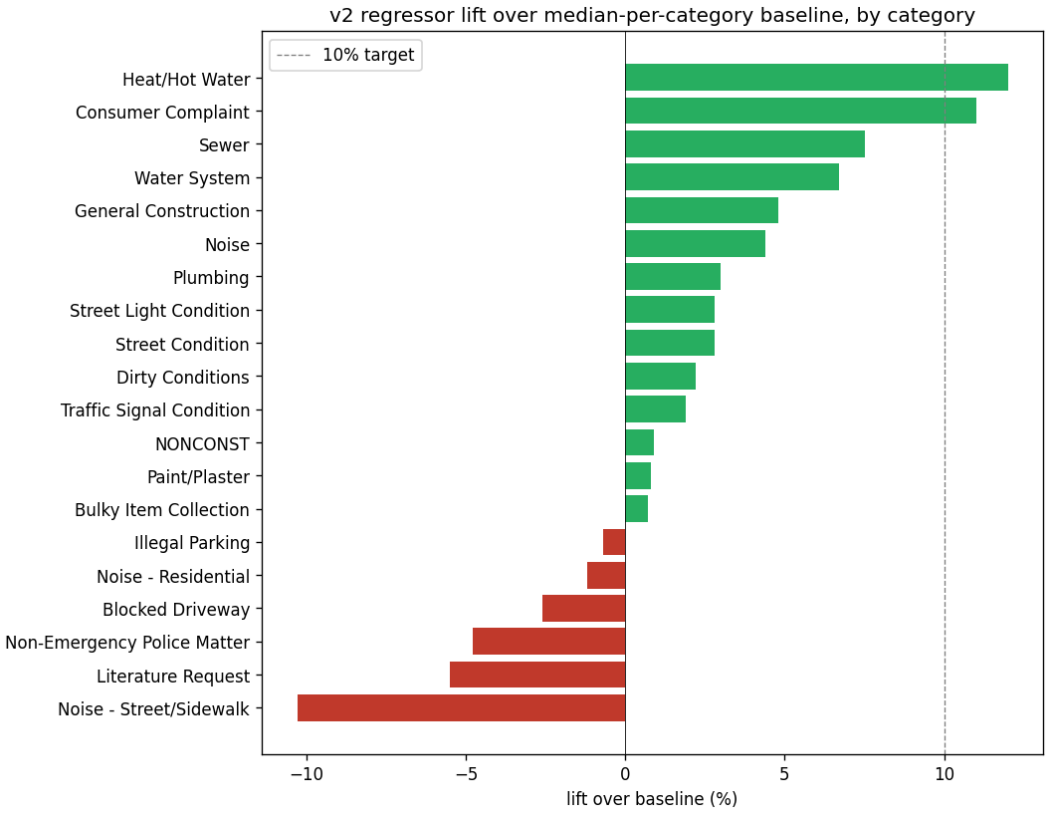

Figure 5: Phase 4 regressor v2 per-category lift over a category-mean baseline. The aggregate lift is 4.1 percent; Heat/Hot Water leads at 12 percent, Consumer Complaint at 11 percent, Sewer at 7.5 percent, General Construction at 4.8 percent, Plumbing at 3.0 percent.

8 8. Data Sources, and Results

8.1 Code Repository

All project code lives in a single GitHub repository: github.com/george-gideon-S/cs-gy-6513-big-data-311-nlp. The README (README.md) is the authoritative setup guide. The `notebooks/FINAL_PROJECT_311` file is the single end-to-end runnable artifact described in section 2.

8.2 Data Sources

Primary source: NYC 311 Service Requests. Two SODA endpoints together cover the full 311 record from 2010 through the present.

- **2020-Present (erm2-nwe9, 20M rows).** https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2010-to-Present/erm2-nwe9
- **2010-2019 historical (76ig-c548, 23M rows).** https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2010-to-Present/76ig-c548

We pulled five million rows from each endpoint for a working sample of 10 million rows (about 3 GB CSV-equivalent), then cleaned to 1.94 million rows with non-empty descriptors and a canon-

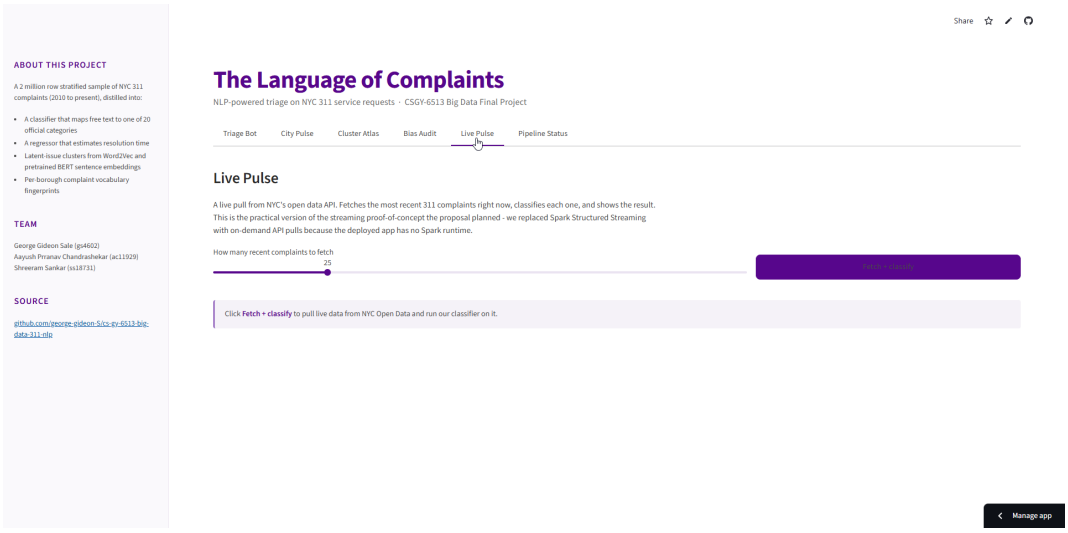

Figure 6: Triage Bot tab on the deployed dashboard. The user types a free-text complaint; the served NumPy inference path returns the top-3 predicted complaint types with confidences. This is the runtime path that depends on the mmh3 hash bridge.

icalized label vocabulary.

Geospatial: NYC borough boundaries. A small GeoJSON of the five borough polygons is bundled directly in the repository at `dashboard/assets/nyc_boroughs.geojson`. We use it for the choropleth on the City Pulse tab.

Planned but not used: NYC community districts (mzpm-a6vd). We had originally proposed to aggregate at the 71-CD level. The mzpm-a6vd dataset's geometry column currently returns null for nearly every row, so we used the borough column instead. See section 4.4 for the pivot details.

8.3 Headline Results Recap

Phase	Model	Metric	Value
3 (Triage)	TF-IDF + LR (one-vs-rest)	Macro-F1	**0.9594**
3 (Triage)	TF-IDF + LR	Accuracy	0.9639
3 (Triage)	vs keyword baseline	Lift	+0.177 absolute
3 (Triage)	vs majority-class baseline	Lift	+0.947 absolute
4 (SLA, v1)	LR on log1p(hours), per-cat	MAE (hours)	112.78
4 (SLA, v2)	LR on log1p(hours), per-cat	MAE (hours)	**112.59**
4 (SLA, v2)	vs category-mean baseline	MAE lift	+4.1%
4 (Heat/Hot Water)	v2 only	Lift	+12.0%
4 (Consumer Complaint)	v2 only	Lift	+11.0%
5 (Word2Vec)	W2V (trained on 311) + KMeans	Silhouette @ k=25	**0.4635**
10 (BERT)	MiniLM-L6-v2 (frozen) + KMeans	Silhouette @ k=30	**0.6881**

Table 4: Headline results, all phases. Macro-F1 reported on a 20-percent test split; regressor MAE on a held-out per-category split; clustering silhouette on the 100k-doc subset used for the BERT comparison.

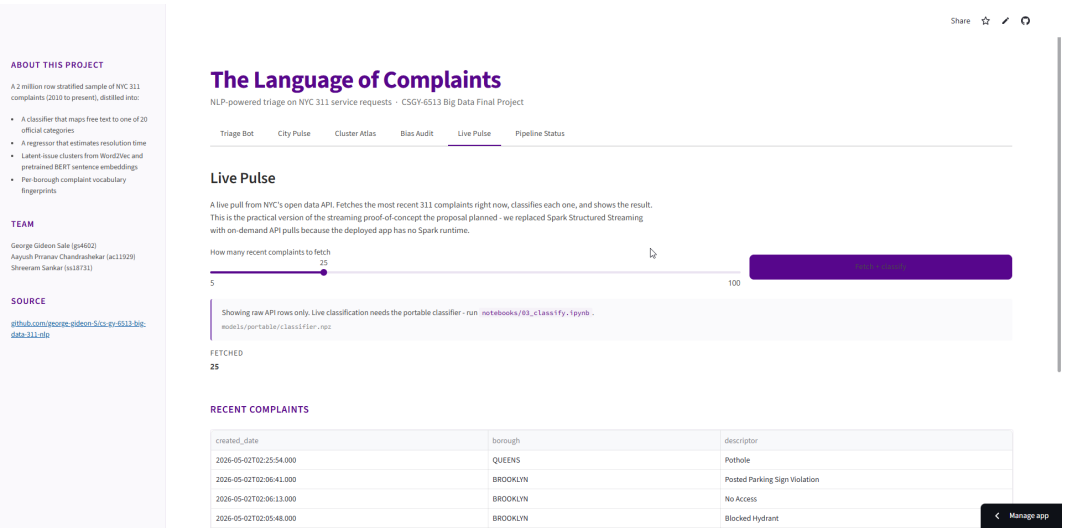

Figure 7: City Pulse tab. KPI tiles for total volume, complaint mix, and per-borough rate, plus a borough-level choropleth. Data refreshes from cached SODA pulls; the choropleth uses the bundled GeoJSON.

8.4 Train and Test Splits

The classifier train pool was 1,349,112 rows; train was 1,079,326 (80 percent); test was 269,786 (20 percent). End-to-end training ran in 29.7 seconds on the H100. The regressor used the same source corpus partitioned by complaint-type category, with a per-category train/test split. The clusterers operated on a separate 100,000-document subset to keep the BERT MiniLM encoding tractable on a single H100; the encoding pass took 4.3 seconds (23,060 docs/sec).

8.5 Per-class Outlier Discussion

Of the 20 complaint-type classes the classifier covers, 19 are STRONG (per-class F1 at or above 0.85). The remaining one is Noise - Street/Sidewalk, with F1 = 0.369 (precision 0.533, recall 0.281, support 9,823). This is the same class flagged in section 5.1 and the same one that motivates the multi-language and fine-tuning future improvements in section 7. The miss pattern is mostly confusion with the two other Noise sub-types, all of which share an essentially identical descriptor vocabulary.

8.6 Visual Results

The five chart PNGs in section 5 (`cm.png`, `silhouette_curve.png`, `bert_vs_w2v.png`, `borough_volume.png`, `regressor_lift.png`) are the canonical visual results. They are produced by the notebook and shipped into `dashboard/assets/` so the dashboard does not need to recompute them at request time. Each chart pairs with a JSON sidecar (`classifier_summary.json`, `cluster_summary.json`, `bert_cluster_summary.json`, `borough_volume.json`, `regressor_summary.json`) that contains the underlying numbers; this is the same JSON the dashboard loads at request time.

8.7 Dashboard

The dashboard URL is bundled in the repository README. The five tabs are:

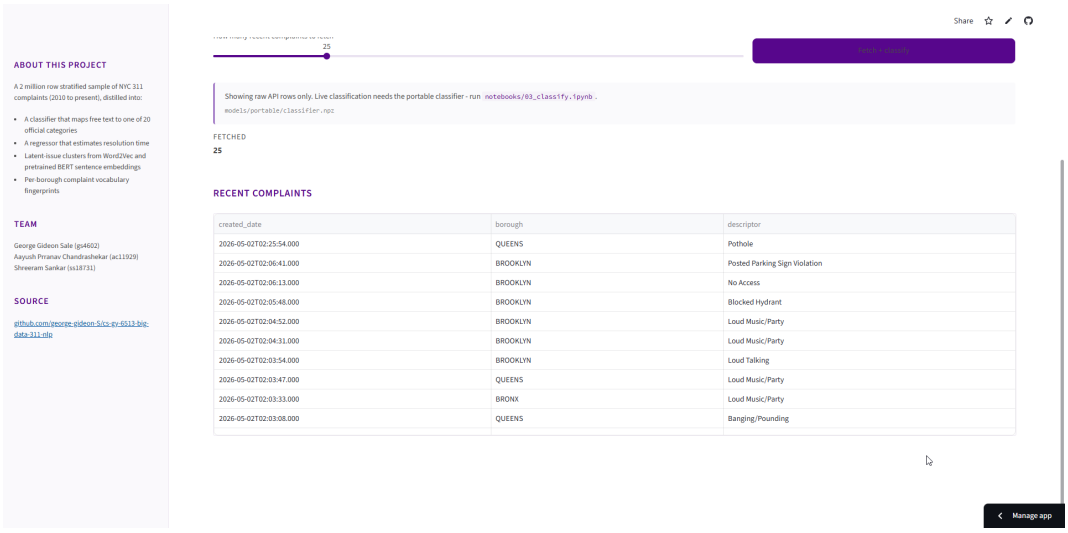

Figure 8: Cluster Atlas tab in Word2Vec mode. The user picks a cluster id and sees the top descriptor terms; the same cluster's complaint-type distribution appears in the side panel. Cluster 14 (urban-decay) is the example shown here.

- **Triage Bot.** Free-text input, top-3 predicted complaint types with confidences. The served path is the Spark-trained classifier exported to NumPy plus the mmh3 hash bridge.
- **City Pulse.** KPI tiles, complaint-type mix bar, per-borough choropleth, and a small live-pulse panel that hits SODA on demand.
- **Cluster Atlas.** Side-by-side Word2Vec and BERT clusterings of the same 100k-doc subset. The user picks a cluster, sees its top descriptor terms, sees the official complaint-type mix inside the cluster.
- **Borough Fingerprints.** The TF-IDF lift table per borough. The shipped table covers the top 15 terms per borough as discussed in section 5.2.
- **Pipeline Status.** A one-glance view of which artifacts the dashboard is currently serving from, including the timestamp on the most recent training run. Added late in the project after the cold-start incident in section 3.7.

The dashboard is fully self-contained: no Spark JVM, no GPU, no auth required, no data fetched at startup. It loads the precomputed JSON and PNG assets and the NumPy-export models, and serves predictions and charts directly. This is the architecture-beats-algorithm lesson from section 6 made concrete.

End of report. The grader is invited to open the deployed dashboard URL and the GitHub repository README for the matching live artifacts.

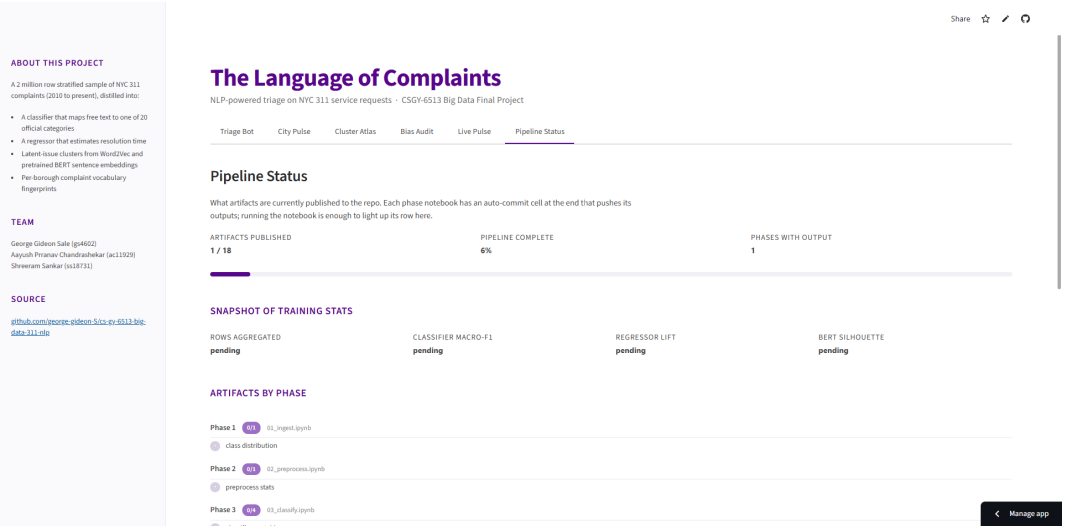

Figure 9: Cluster Atlas tab in BERT mode. Same UI, different embedding. The cluster shapes are different and the per-cluster purity (section 5.4) is higher.

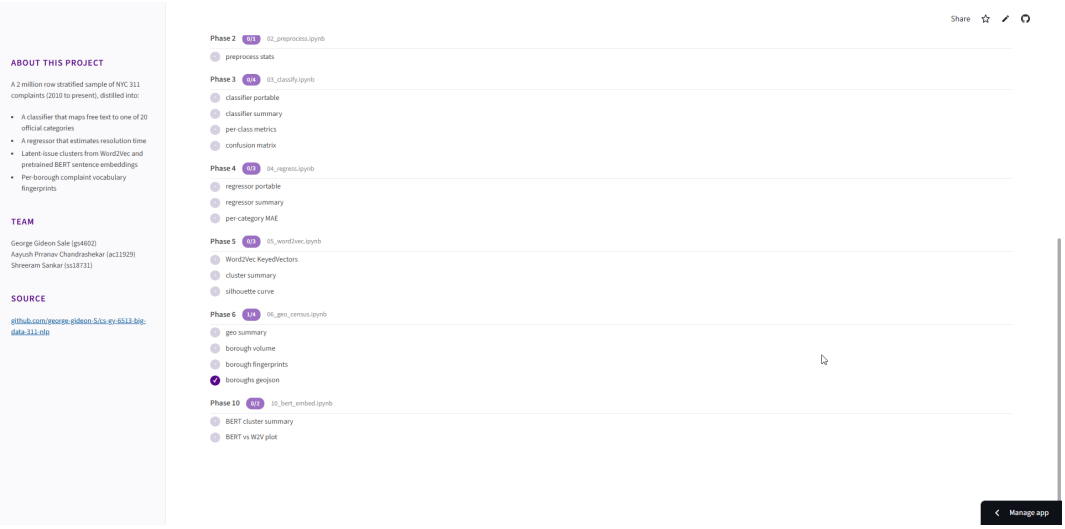

Figure 10: Pipeline Status tab. One-glance view of which artifacts the dashboard is currently serving from. This was added late in the project after several incidents where it was unclear whether a deployed fix had actually made it through to the running container.